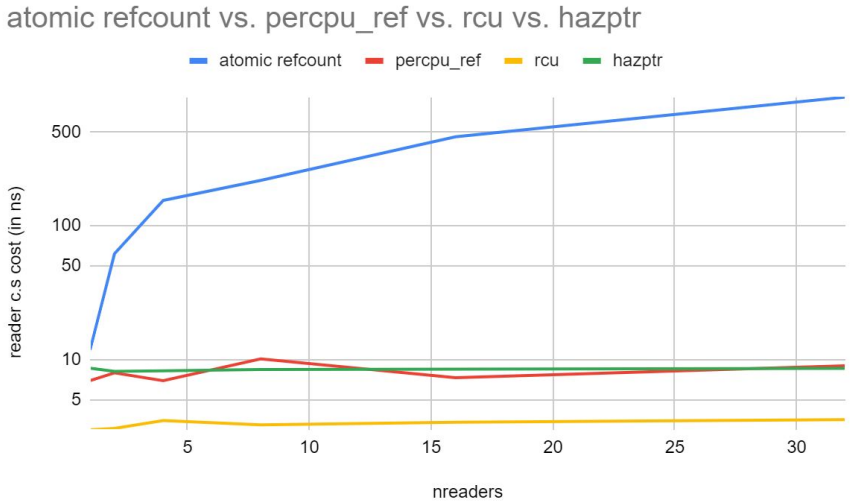
A (Simple) Hazard Pointer Update

Boqun Feng 冯博群 (Netflix Inc.)
Paul E. McKenney (Meta)

Previously on

- LPC 2024 talk - “Hazard pointers in Linux kernel”
- [RFC PATCH 0/4] Add hazard pointers to kernel
  - Address dependency issue
  - Actual usage



High level view

Reuse Paul’s slides at ALPSS

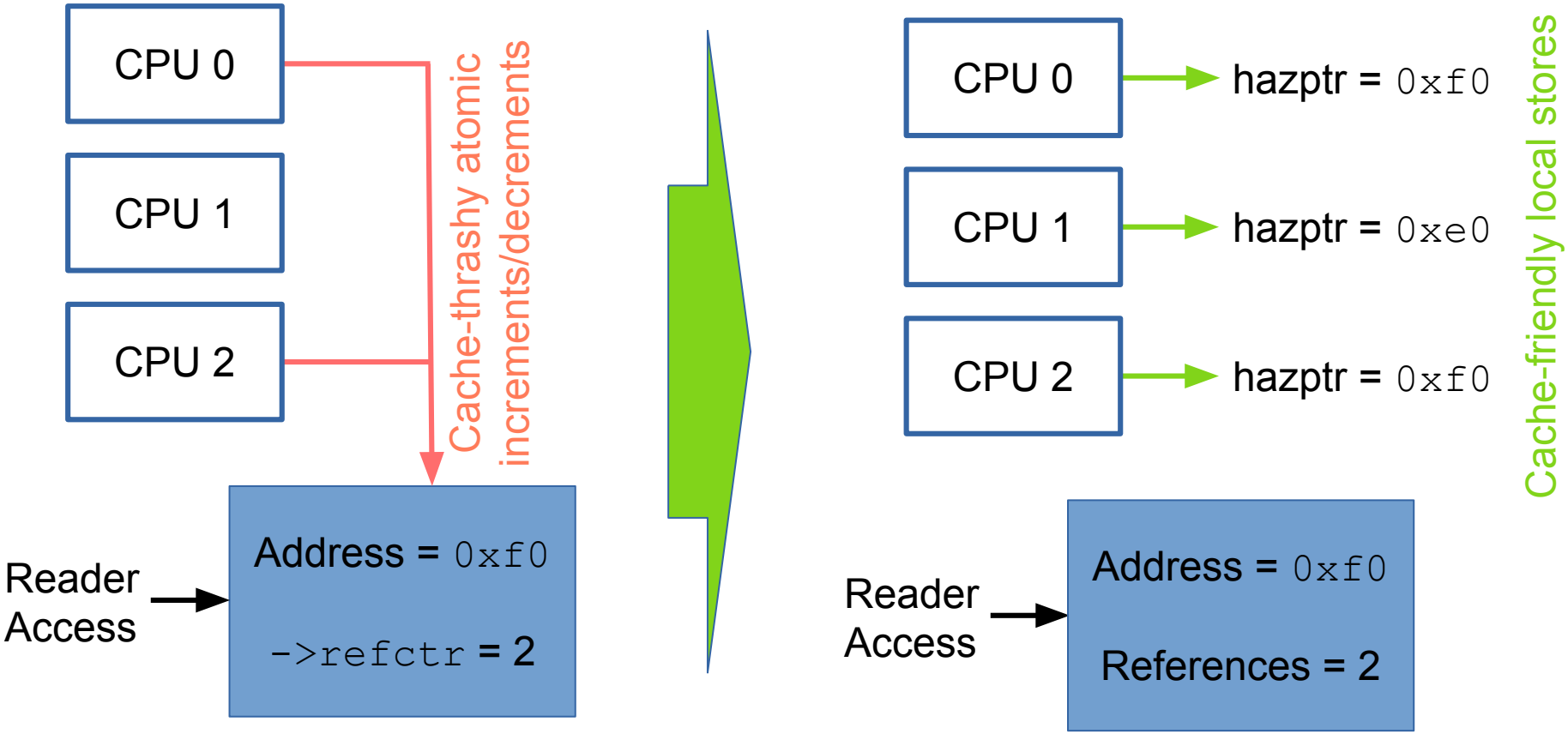
Hazard Pointers

- Instead of a counter, count the number of “hazard pointers” referencing a given object
  - Hazard pointers can be local to a CPU or task, thus avoiding expensive cache misses
  - In the absence of updaters, hazard-pointer readers are fast and scalable
  - A given hazard pointer might reference any object

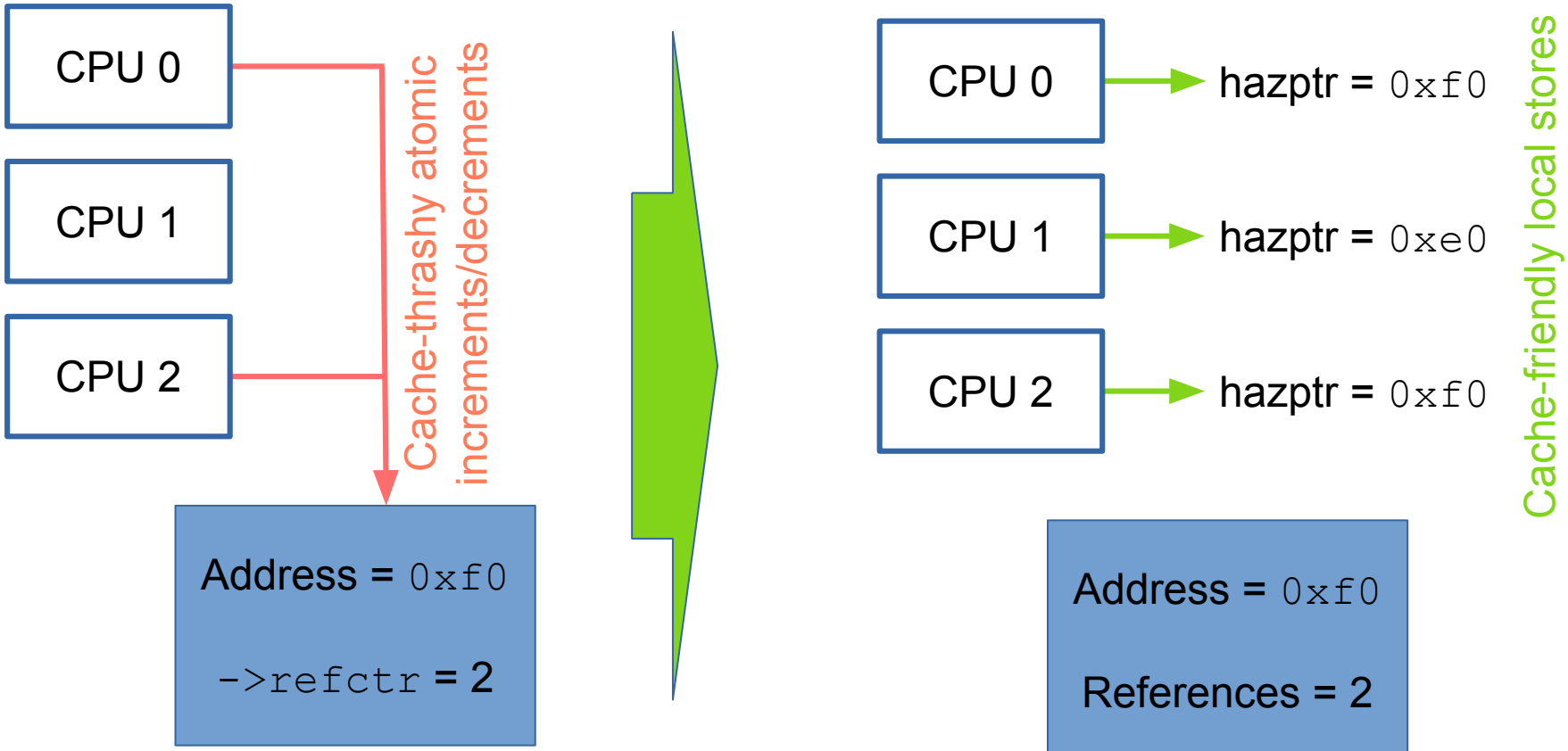
What Exactly is a Hazard Pointer?

- A hazard pointer has two components:
  - A raw pointer (void *) to the protected object
  - Overhead fields needed to track all of the hazard pointers in some search data structure
    - The exact type and number of fields depends on the choice of search data structure

Hazard Pointers, Graphical View

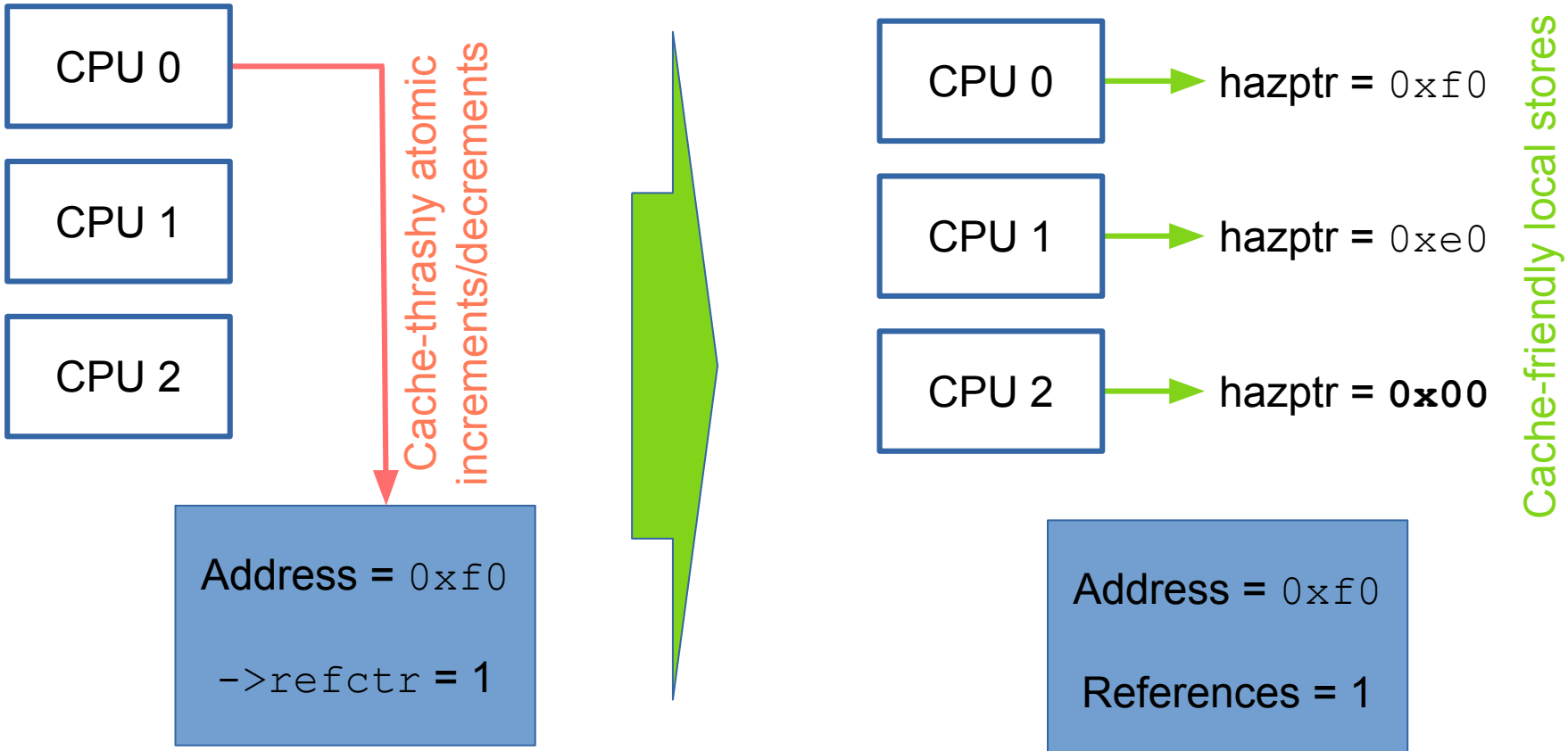


Hazard Pointers, Graphical View



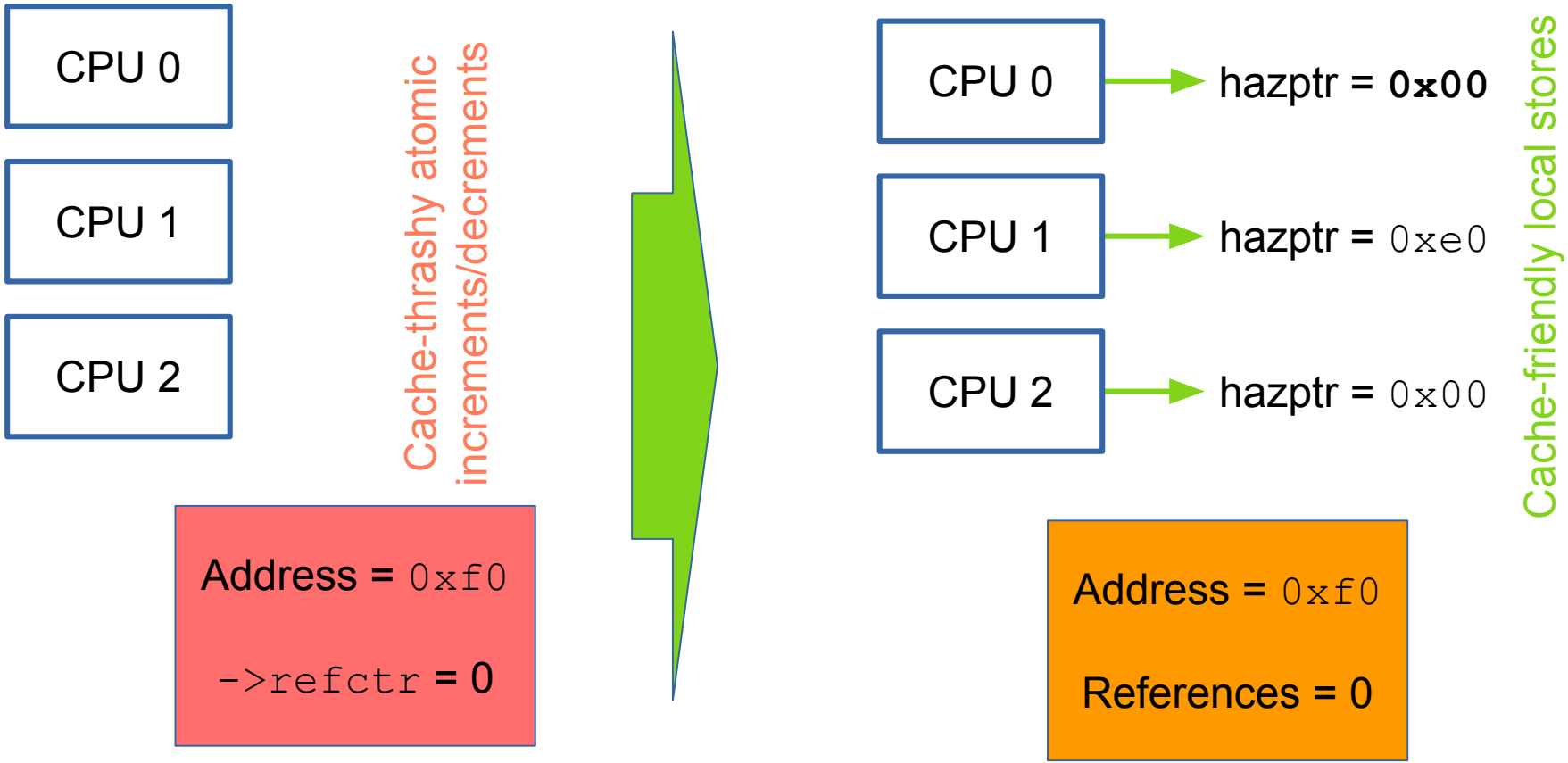
Remove Reader Access

Hazard Pointers, Graphical View



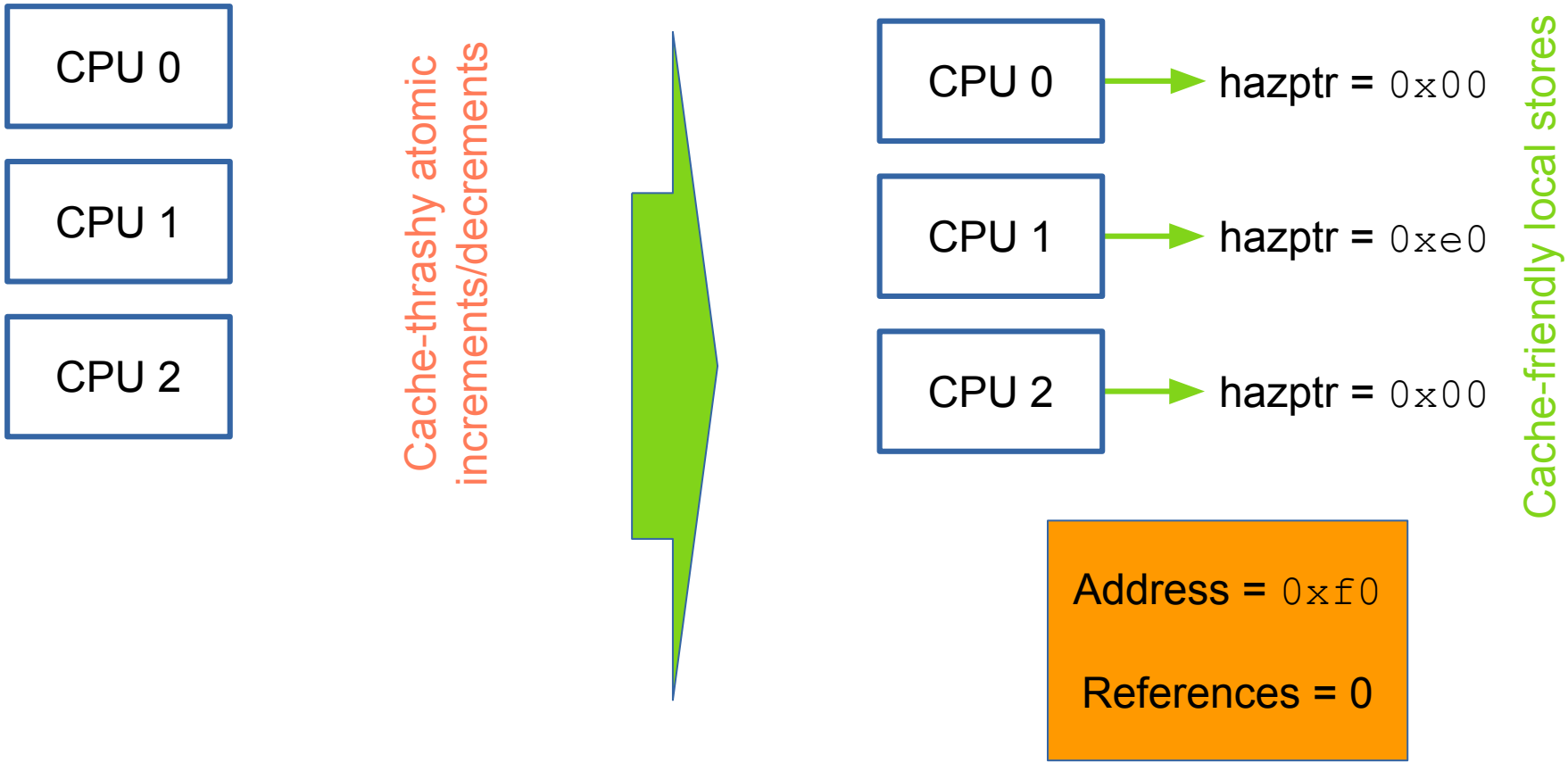
CPU 2 drops reference

Hazard Pointers, Graphical View



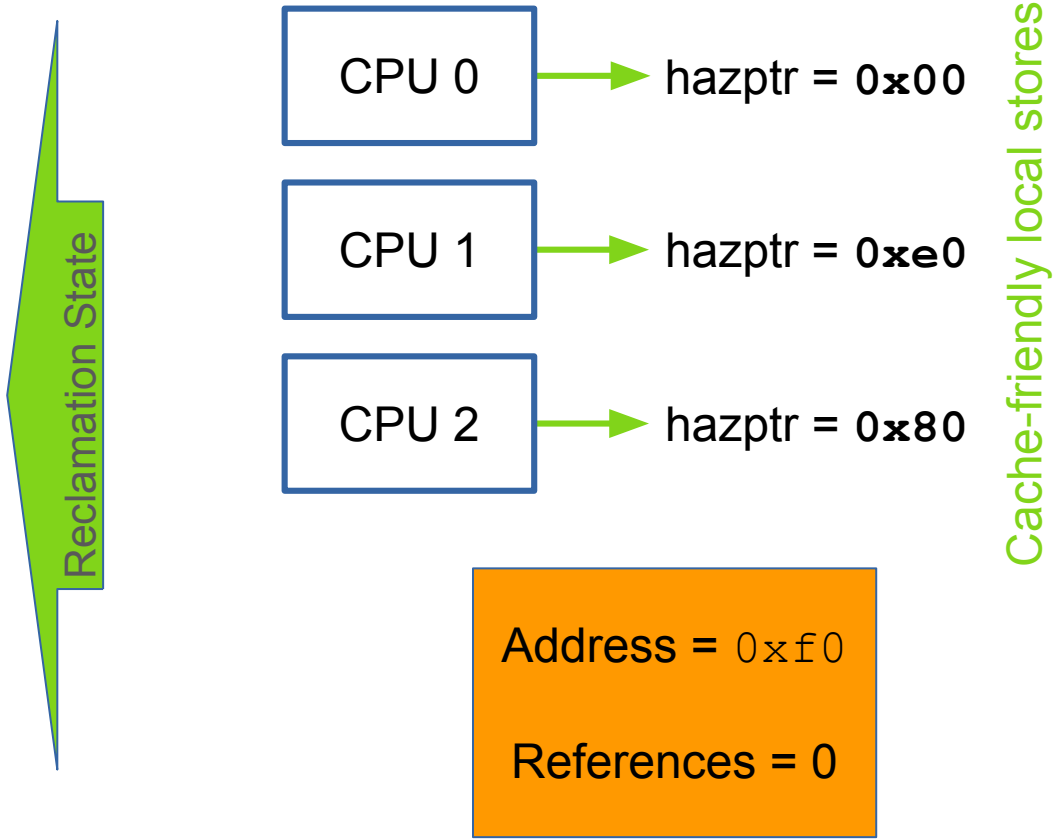
Can immediately free in `refctr` case, but need `call_hazptr()` for hazard pointers

Hazard Pointers, Graphical View



Reference-counter frees immediately, but hazard pointers defer free (optimization)

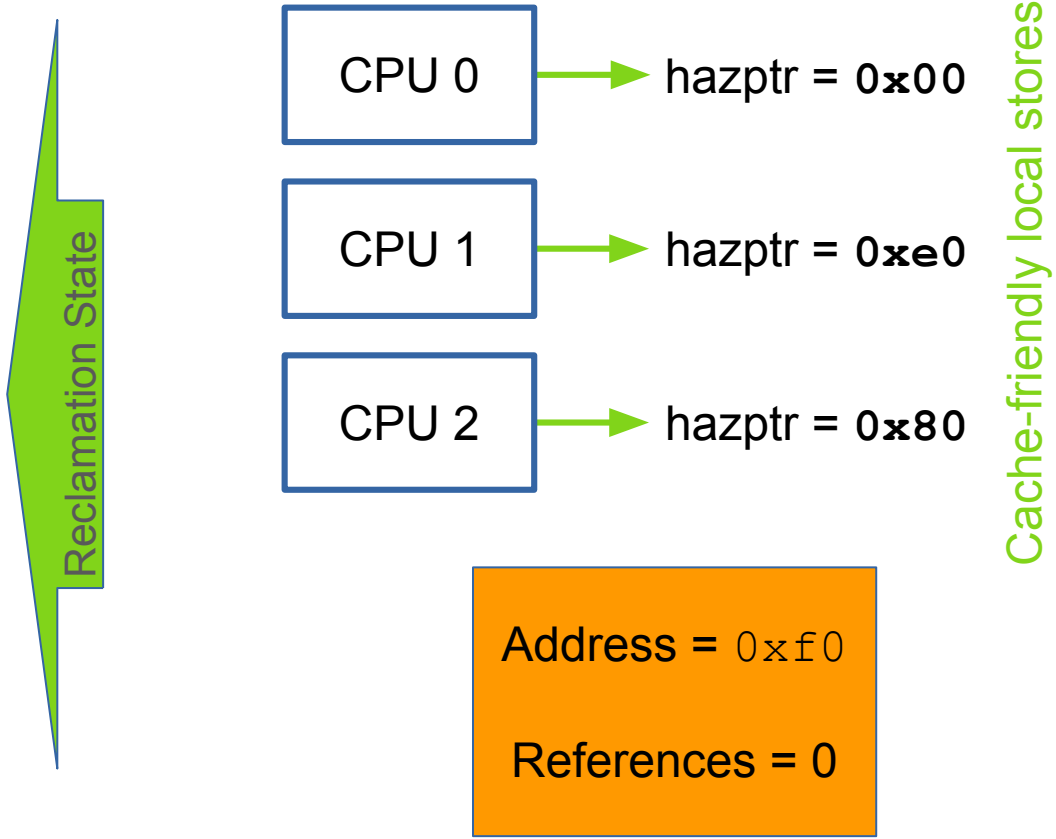
Hazard Pointers, Graphical View



Repurpose left-hand portion of slide for hazard pointer reclamation state

Hazard Pointers, Graphical View

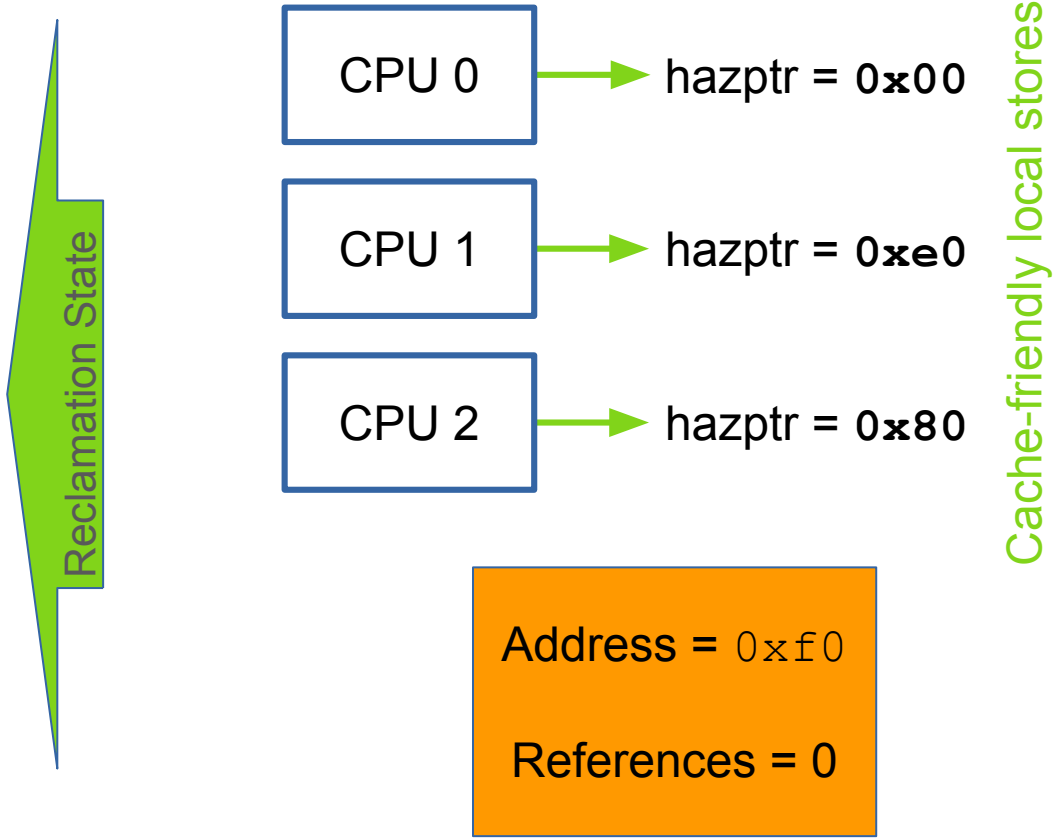
- Reclaim list
  - `0xf0`



Step 1: Gather to-be-reclaimed objects passed to `kfree_hazptr()`

Hazard Pointers, Graphical View

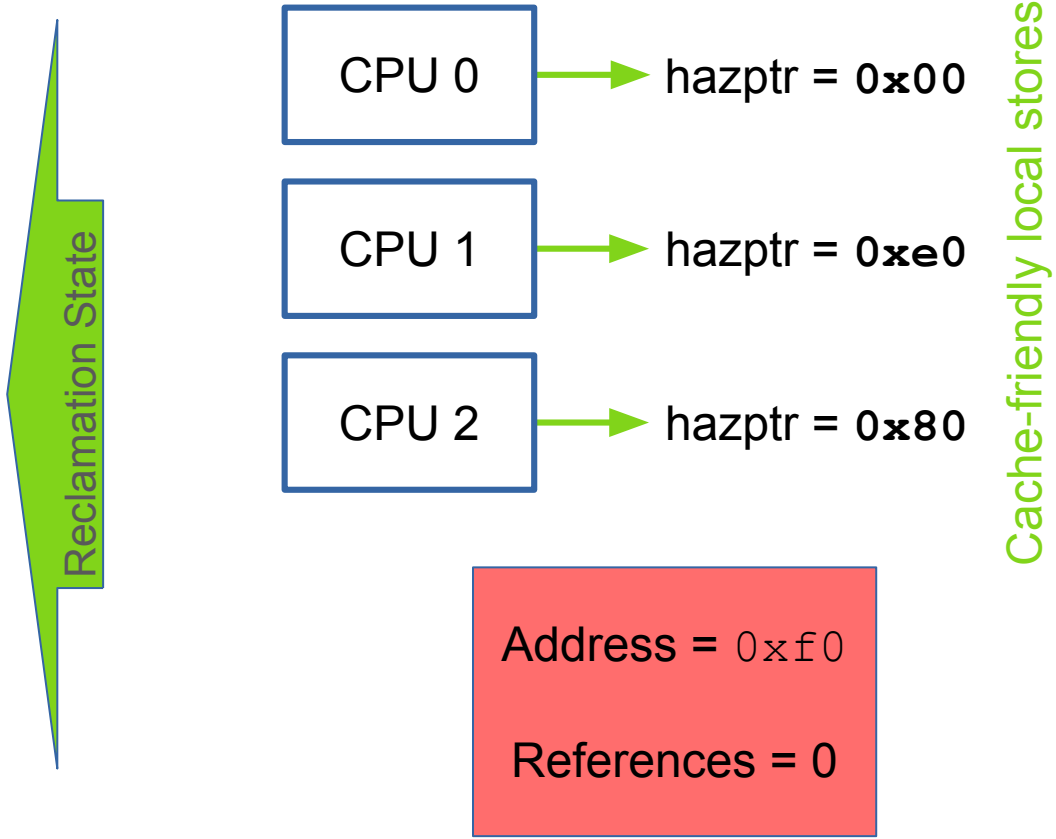
- Reclaim list
  - `0xf0`
- Hazard pointers
  - `0x00`
  - `0xe0`
  - `0x80`



Step 2: Gather hazard pointers

Hazard Pointers, Graphical View

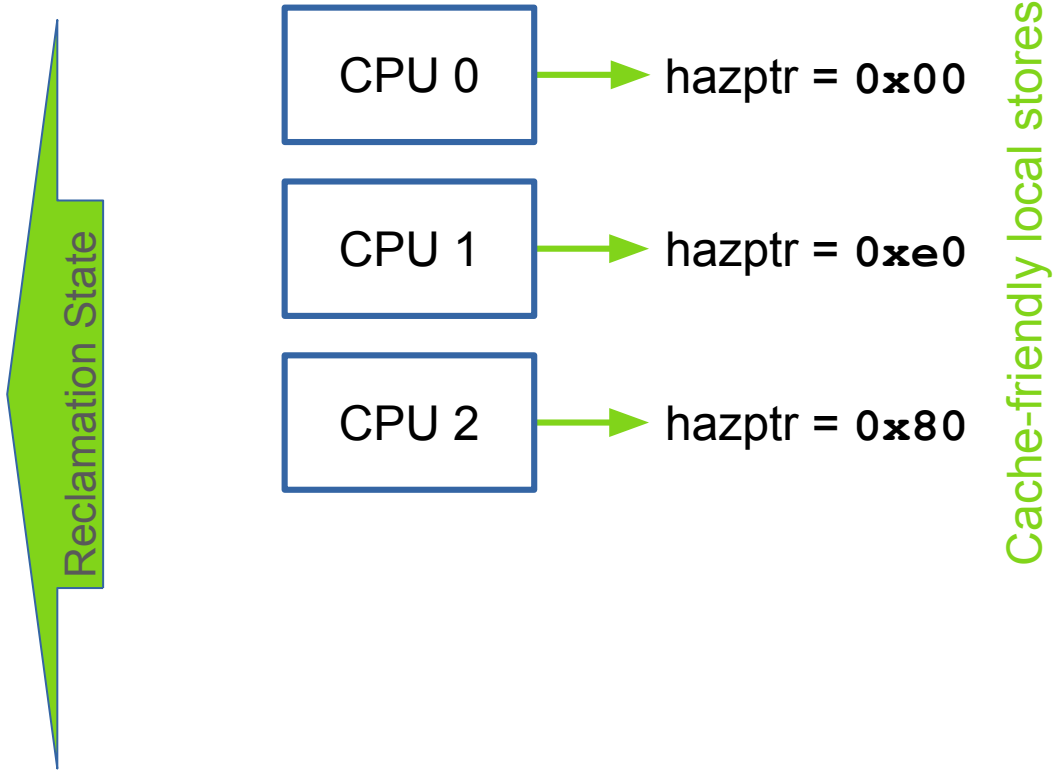
- Reclaim list
  - `0xf0`
- Hazard pointers
  - `0x00`
  - `0xe0`
  - `0x80`



Step 3: Determine which to-be-reclaimed objects are unreferenced by hazard pointers

Hazard Pointers, Graphical View

- Reclaim list
  - `0xf0`
- Hazard pointers
  - `0x00`
  - `0xe0`
  - `0x80`



Step 4: Reclaim (free) identified objects, including the one at address `0xf0`

Hazard Pointers, Good and Bad

- **Bad**
  - **Can’t tell exactly when the number of hazard pointers referencing an object goes to zero, delaying reclamation**
  - **Need hazard-pointer allocation and deallocation**
  - **In general, cannot unconditionally traverse a hazard-pointer-protected pointer (similar to reference counting)**
- **Good**
  - **Hazard-pointer acquire/release scalable & reasonably fast**
  - **Long-duration readers block reclamation of only those objects that they are actually using at that time**

Which to use???

Property	Hazard Pointers	RCU
Readers	Fast and scalable	Faster and scalable
Reader Memory Overhead	Pointer/reader/object + updater traversal pointer(s)	None
Reader duration	Can be very long	User must bound duration
Traversal retries?	If object deleted	Never
Reclamation delay	Batching optimizations	Batching optimizations plus maximum reader duration
Updater Memory Overhead	Maybe hazptr_head structure	Maybe rcu_head structure

API: hazard pointer vs RCU

- Allocate hazard pointer slots

<hazard pointer>

<RCU>

```
init_hazptr_context(ctx);
hazptr_t *hptr = hazptr_alloc(ctx);
```

API: hazard pointer vs RCU

- Reader side

<hazard pointer>

```
if (p = hazptr_tryprotect(hptr, gp)) {
     <use p>
     hazptr_clear(hptr);
}
```

<RCU>

```
rcu_read_lock();
p = rcu_dereference(gp);
<use p>
rcu_read_unlock();
```

API: hazard pointer vs RCU

- Updater side: `synchronize_hazptr(ptr)` only waits for the readers accessing `ptr`

<hazard pointer>

```
ptr = READ_ONCE(gp);
WRITE_ONCE(gp, NULL);
synchronize_hazptr(ptr);
kfree(ptr);
// or call_hazptr() or kfree_hazptr()
```

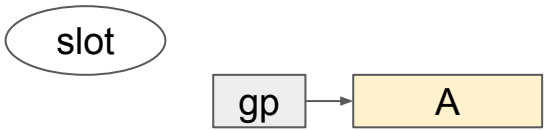
<RCU>

```
ptr = READ_ONCE(gp);
rcu_assign_pointer(gp, NULL);
synchronize_rcu();
kfree(ptr);
// or call_rcu() or kfree_rcu()
```

Synchronization (or Memory ordering magic) behind

Recap: Hazard pointers slot

- Hazard pointer slots
  - Can store one pointer value
  - Updaters can access all the slots



Recap: hazptr_tryprotect(): double reads

```
1: tmp1 = READ_ONCE(gp); // 1st read: snapshot the global pointer value
2: WRITE_ONCE(*slot, tmp1); // store the value into a hazard pointer slot
3: smp_mb();
4: tmp2 = READ_ONCE(gp);  // 2nd read: re-snapshot the global pointer value
5: if (ptr_eq(tmp1,tmp2)) {
    return tmp2;
} else {
5b: WRITE_ONCE(*slot, NULL); // reset hazard pointer slot
    <continue step 1 or abort>
}
```

Recap: hazptr_clear()

```
1: smp_store_release(slot, NULL);
```

Recap: synchronize_hazptr()

```
// 1: todo = READ_ONCE(gp);
// 2: WRITE_ONCE(gp, NULL); // unpublish todo
3: smp_mb();
4: foreach slot {
  4.1: ptr = smp_load_acquire(slot); // fetch the pointer from a reader
  4.2: if (ptr == todo) {
           <continue to step 2.1>*
       }
}
```

Recap: Hazard pointers (updater publish)

```
1: <initialize ptr->field>
2: smp_store_release(&gp, ptr);
```

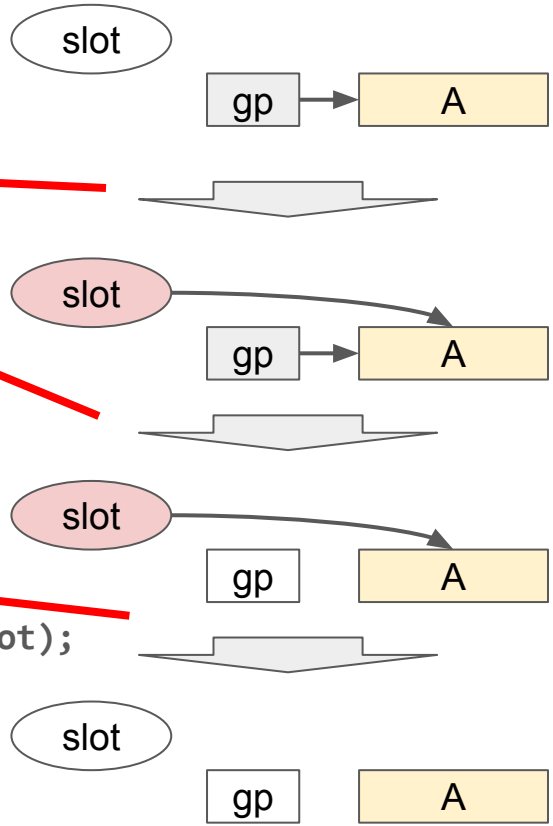
Recap: Hazard pointers (synchronization case #1)

```
<reader>                          <updater>
1: tmp1 = READ_ONCE(gp);
2: WRITE_ONCE(*slot, tmp1);

                                  // 2: WRITE_ONCE(gp, NULL);
                                  ...

3: smp_mb();
4: tmp2 = READ_ONCE(gp);
5: if (ptr_eq(tmp1,tmp2)) { // false, reader will try again
} else {
5b:  WRITE_ONCE(*slot, NULL);


                                  4.1: ptr = smp_load_acquire(slot);
}
```



Recap: Hazard pointers (synchronization case #2)

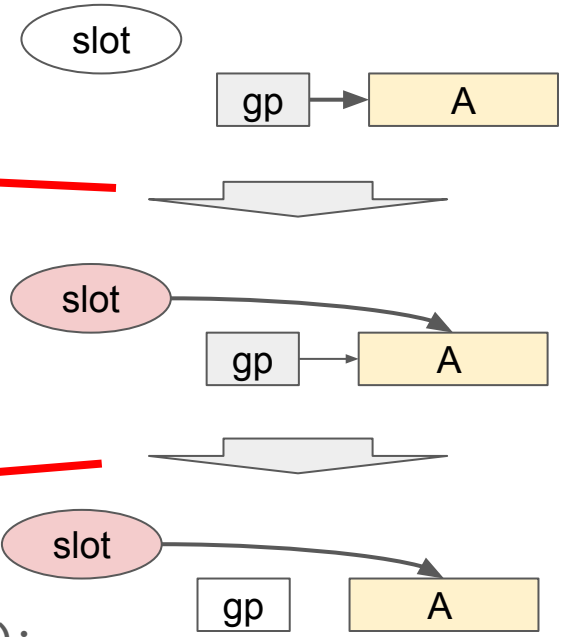
```
<reader>
1: tmp1 = READ_ONCE(gp);
2: WRITE_ONCE(*slot, tmp1);
3: smp_mb();
4: tmp2 = READ_ONCE(gp);
5: (ptr_eq(tmp1,tmp2)) {
     return tmp2;



}
```

```
<updater>

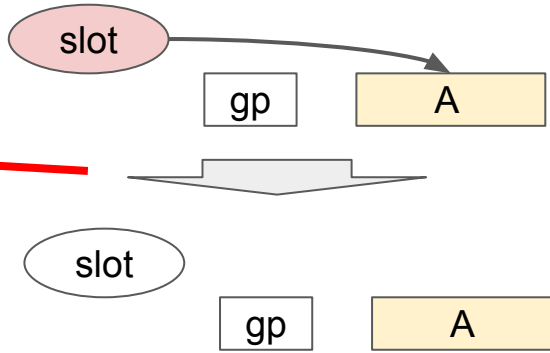
// 1: todo = READ_ONCE(gp);
// 2: WRITE_ONCE(gp, NULL);
3: smp_mb();
4: foreach slot {
  4.1: ptr = smp_load_acquire(slot);
  4.2: if (ptr == todo) {
     <continue to step 4.1>
   }
}
```



Recap: Hazard pointers (synchronization case #3)

```
<reader>                    <updater>

smp_store_release(slot, NULL);
                            4.1: ptr = smp_load_acquire(slot);
                            4.2: if (ptr == todo){
                                 ...
                                 }
```



Address dependency issue

```
<reader>

1: tmp1 = READ_ONCE(gp);

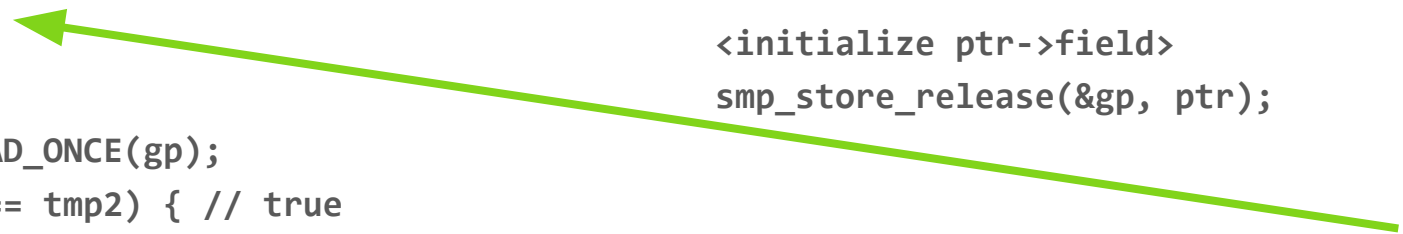
2: WRITE_ONCE(*slot, tmp1);
3: smp_mb();


4: tmp2 = READ_ONCE(gp);
5: if (tmp1 == tmp2) { // true
     <uses tmp2->field> // <- compiler can optimize to tmp1->field, and can be reordered
}
```

```
<updater>
smp_store_release(&gp, ptr);

<free ptr and reuse the same ptr>


<initialize ptr->field>
smp_store_release(&gp, ptr);
```



Spotted by Jonas Oberhauser

Address dependency issue: Solution

```
static __always_inline
int ptr_eq(const volatile void *a, const volatile void *b)
{
     OPTIMIZER_HIDE_VAR(a);
     OPTIMIZER_HIDE_VAR(b);
     return a == b;
}
```

Developed by Mathieu Desnoyers

Case study

`lockdep_unregister_key()` “performance” problem

- Each dynamic lockdep key needs to be registered before use and unregistered after use.
- The registration keys are in a RCU protected hash list.
- lockdep_unregister_key() needs to call synchronize_rcu() to wait for all accesses to the hash list
  - But it also means it will wait for all RCU read side critical sections!
  - Breno Leitao proposed using `synchronize_rcu_expedited()` which shows 10x speedup but at the cost of 10x IPIs.
- OK, maybe hazptr?

2 failures we need to handle when using hazard pointers

- Protect failure, i.e. reader observes the changes from updaters.
  - Remember it's called hazptr_`tryprotect()`?
- Allocation failure, the allocation of the hazard pointer slots may fail.

Handle protection failure

For the lockdep case, it’s easy to handle, because we can just protect the whole list.

```
<in is_dynamic_key()>
rcu_read_lock();
hlist_for_each_entry_rcu(list) {
   <return true if found>
}
rcu_read_unlock();


<updater>
hlist_del_rcu();
synchronize_rcu();
```



```
<in is_dynamic_key()>
hazptr_acquire(..., list);
hlist_for_each_entry_rcu() {
   <return true if found>
}
hazptr_clear(...);


<updater>
hlist_del_rcu();
synchronize_hazptr(list);
```

Handle protection failure: what's `hazptr_acquire()`?

```
hazptr_acquire(slot, gp): // The first part of hazptr_tryprotect()
  void *ptr = READ_ONCE(gp);
  WRITE_ONCE(slot, ptr);
  smp_mb();
```

Handle protection failure

```
<in is_dynamic_key()>

haztpr_acquire(.., list):

  WRITE_ONCE(*slot, list);


  smp_mb();

hlist_for_each_entry_rcu() {
   <return true if found>
}
hazptr_clear(...);
```

```
<in lockdep_unregister_key()>


hlist_del_rcu(..);

synchronize_hazptr(list); // wait
```

Handle protection failure

```
<in is_dynamic_key()>
haztpr_acquire(.., list):

  WRITE_ONCE(*slot, list);
  smp_mb();
hlist_for_each_entry_rcu() {
   <return true if found> // will observe the deletion
}
hazptr_clear(...);
```

```
<in lockdep_unregister_key()>
hlist_del_rcu(..);
synchronize_hazptr(list);
```

Handle allocation failure

- We could allocate the slots dynamically (per-CPU, per-task, on-demand).
  - But what if the allocation fails.
- Or we can avoid handling this:
  - We could use a static per-CPU hazard slot and since each time that the hash list is visited, we also have the interrupt disabled.

Simple Hazard Pointers

Simple hazard pointers

- If
  - we only use hazard pointers under preemption disabled, and
  - We **mostly** only reference one object a time
- Then
  - We have a per-CPU hazard pointer slot.

Simple hazard pointers: nested readers

- But `is_dynamic_key()` can be called in an NMI

```
shazptr_acquire(list1):
  WRITE_ONCE(*this_cpu_slot, list1);
  <in NMI>
  shazptr_acquire(list2):
    WRITE_ONCE(*this_cpu_lost, list2); ???
```

Simple hazard pointers: WILDCARD

- WILDCARD means the slot is protecting all the addresses. It'll block all the `synchronize_shazptr()`s.

```
shazptr_acquire(list1):
  WRITE_ONCE(*this_cpu_slot, list1);
  <in NMI>
  shazptr_acquire(list2):
    WRITE_ONCE(*this_cpu_lost, WILDCARD);
```

Simple hazard pointers: WILDCARD

```
guard1 = shazptr_acquire(list1):
  WRITE_ONCE(*this_cpu_slot, list1);
  <in NMI>
  guard2 = shazptr_acquire(list2):
    WRITE_ONCE(*this_cpu_lost, WILDCARD);
  shazptr_clear(guard2):
    <do nothing>
shazptr_clear(guard1):
  smp_store_release(this_cpu_slot, NULL);
```

Simple hazard pointers: WILDCARD

- As long as the readers are perfectly nested, then WILDCARD can support as many readers as possible.
- The cost is when WILDCARD is activated, the CPU blocks all synchronize_shazptr(), but when that becomes a problem, maybe it’s time to revisit to use dynamic allocation of hazptr.

Single shazptr domain vs Multiple shazptr domains

- We can have multiple shazptr “domain”s (basically multiple isolated sets of per-CPU slots), but
  - Hazard pointers itself already uses address to separate the domain of blocking, different shazptr “domain”s are just redundant.
  - Single shazptr domain provides the opportunity of optimization for a non-busy-wait version of synchronize_shazptr() because the hazptr slot scan results can be reused.

Implementation and Evaluation

Implementation

- Last patchset:
  - https://lore.kernel.org/lkml/20250625031101.12555-1-boqun.feng@gmail.com/
- Last development branch:
  - https://git.kernel.org/pub/scm/linux/kernel/git/boqun/linux.git/log/?h=shazptr-v2

lockdep_unregister_key() performance

- On par with synchronize_rcu_expedited() solution
  - 13.195s -> 0.135s in Breno's test
    - time /usr/sbin/tc qdisc replace dev eth0 root handle 0x1234: mq
- Less IPIs
  - IPI rate with the shazptr solution: ~14 per second per core.
  - IPI rate with synchronize_rcu_expedited(): ~140 per second per core.

Micro-benchmark vs RCU

Updater side (GP duration in us)

	Avg	Min	Max	p50	p90	p99
RCU	7470.02	3981.6	142228	6002.73	7008.93	10015
shazptr	0.9	0.2	3049.05	0.585	1.656	3.872
shazptr(non-busy-wait)	467.861	92.913	5775.46	440.691	460.623	650.068
shazptr(wildchard)	599.569	1.432	6727.53	582.631	781.704	1160.26
shaptr(wc + non-busy-wait)	460.466	303.546	4126.94	424.334	482.637	600.214

Micro-benchmark vs RCU

- Reader side (old data from hazptr patchset) cost in ns

	1 thread	8 threads	32 threads
RCU	2.97923	3.243	3.55077
hazptr	8.5991	8.40443	8.5762
SRCU	16.7754	22.4807	20.2406